

VITyarthi – Build Your Own Project

ROCK PAPER SCISSORS GAME

Project Report

Student Name	
Registration Number	
Course / Subject	
Academic Year	2026–27

A simple Python command-line project demonstrating functions, conditions, loops, dictionaries, input validation and modular programming.

1. Introduction

Rock Paper Scissors is a simple game where a player chooses rock, paper or scissors and the computer makes its own choice. This project was developed as a Python programming project to apply basic programming concepts in a complete working application.

The project is command-line based. It includes multiple rounds, three difficulty levels, score tracking, game history and input validation. The code is divided into separate Python files so that different parts of the game have separate responsibilities.

2. Problem Statement

A basic game can be used as a practical way to apply programming concepts. The problem addressed by this project is to create a working Rock Paper Scissors game in Python where the user can play against the computer, receive round results and keep track of the overall game.

3. Objectives

- Create a working Rock Paper Scissors game using Python.
- Allow the player to select a difficulty level and number of rounds.
- Determine the result of each round correctly.
- Track wins, losses and draws.
- Store and display game history.
- Validate user input and handle invalid choices.
- Practice modular programming by separating the project into multiple Python files.

4. Functional Requirements

No.	Requirement
1	Select Easy, Medium or Hard difficulty.
2	Enter the number of rounds.
3	Enter rock, paper or scissors.
4	Validate the player's input.
5	Generate the computer's choice.
6	Determine the winner of each round.
7	Update wins, losses and draws.
8	Store the result of each round in history.
9	Display the final score.
10	Display game history.

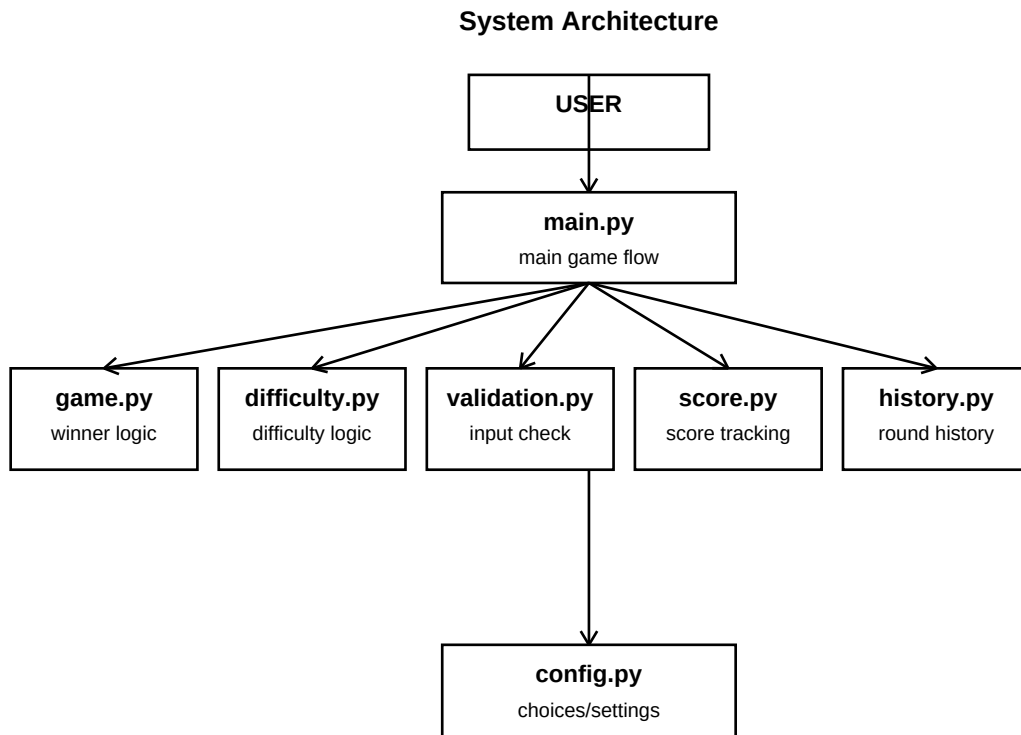
5. Non-Functional Requirements

Requirement	Description
Usability	The game should be simple to understand and operate from the command line.

Performance	A round should produce its result without unnecessary delay.
Reliability	Invalid choices should be handled without stopping the game.
Maintainability	Separate modules make individual parts easier to modify.
Resource efficiency	The small command-line application should require basic computer resources.

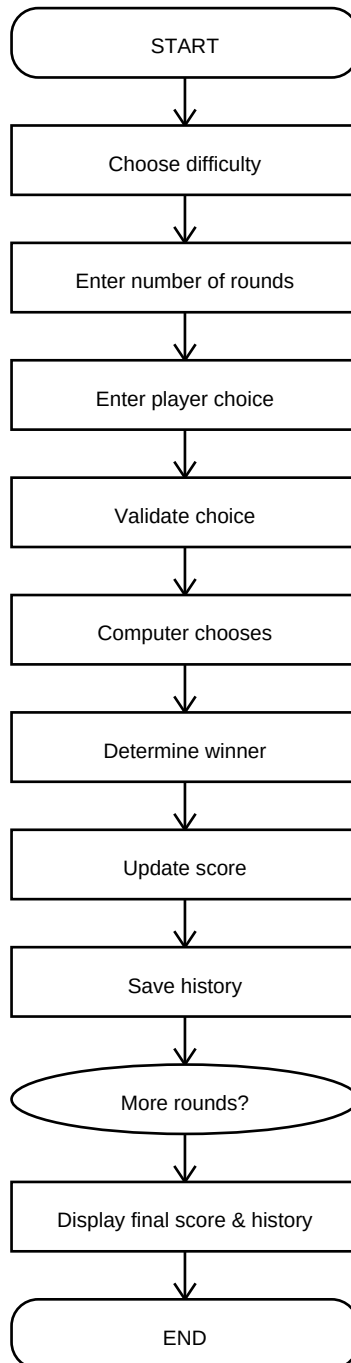
6. System Architecture

The project uses a simple modular architecture. The user interacts with `main.py`, which controls the game flow and calls the other modules.

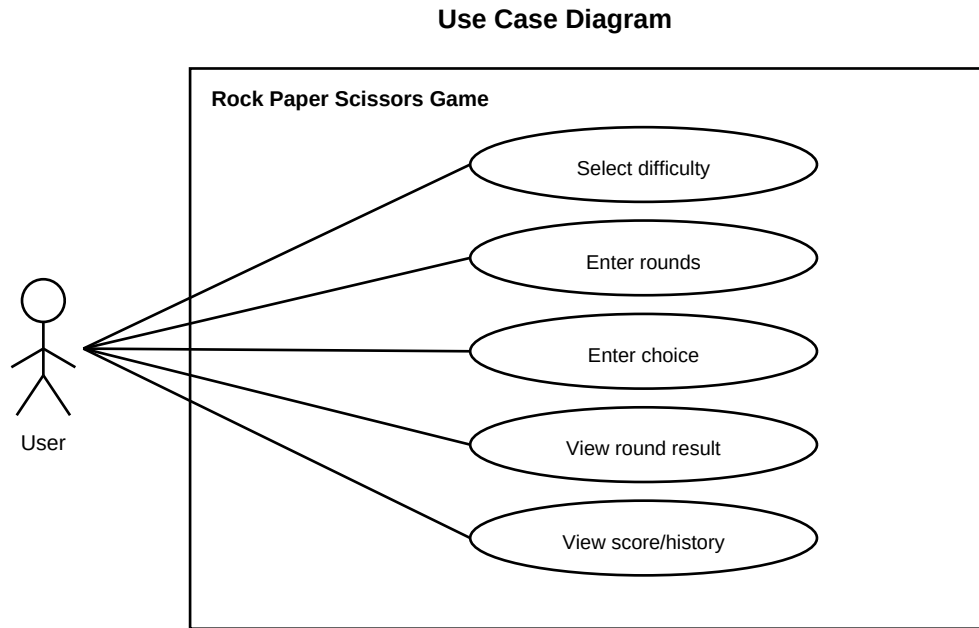


Module roles: `game.py` determines the winner; `difficulty.py` controls computer choices; `validation.py` checks input; `score.py` stores the score; `history.py` stores and displays round history; `config.py` stores common choices and difficulty settings.

7. Game Workflow

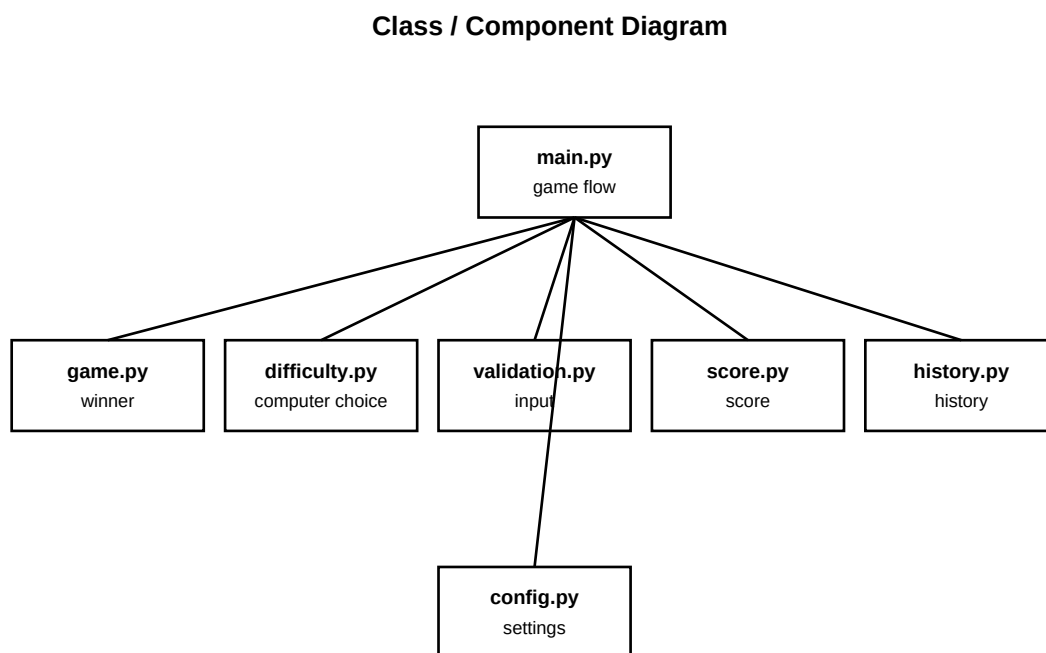


8. Use Case Diagram



The main actor is the User. The user selects difficulty, enters rounds and choices, and views the round result, final score and game history.

9. Class / Component Diagram



10. Design Decisions and Rationale

The project uses separate Python modules instead of putting every part of the game in one file. This keeps responsibilities separated and makes the code easier to understand.

Decision	Reason
Separate game logic	Winner calculation is kept in game.py so it is separate from input and display code.
Separate difficulty logic	Computer choice behavior is handled by difficulty.py.
Dictionary for score	Wins, losses and draws can be stored using simple named keys.
List for history	Each completed round can be added to a list and displayed later.
Input validation	Invalid choices are rejected before the round continues.

11. Implementation Details

The project is written in Python and can be run from Command Prompt using `py main.py`.

File	Purpose
main.py	Controls the complete game flow, user input, rounds, score updates and output.
config.py	Stores game choices and difficulty levels.
game.py	Contains the winner determination function.
difficulty.py	Contains the computer choice logic for the difficulty levels.
score.py	Stores wins, losses and draws.
history.py	Stores completed rounds and displays game history.
validation.py	Checks whether the player's choice is valid.
Rock_Paper_Scissors.ipynb	Notebook version of the game.

12. Screenshots and Results

The following screenshots were captured while running the actual project through Command Prompt.

```
C:\Windows\System32\cmd.e  X  +  v

2. Medium
3. Hard
Enter your choice: 1
Difficulty: Easy

Enter number of rounds: 3

Round 1
Enter rock, paper, or scissors: rock
Your choice: rock
Computer choice: rock
Result: draw

Round 2
Enter rock, paper, or scissors: rock
Your choice: rock
Computer choice: paper
Result: computer wins

Round 3
Enter rock, paper, or scissors: paper
Your choice: paper
Computer choice: scissors
Result: computer wins

===== FINAL SCORE =====
Wins: 0
Losses: 2
Draws: 1

===== GAME HISTORY =====
Round 1 : You = rock | Computer = rock | Result = draw
Round 2 : You = rock | Computer = paper | Result = computer wins
Round 3 : You = paper | Computer = scissors | Result = computer wins

C:\Users\yasht\OneDrive\Desktop\rock paper scissors project>
```

Figure 1: Three-round Easy difficulty test. The final score was 0 wins, 2 losses and 1 draw, and the game history was displayed.

```
C:\Windows\System32\cmd.e  X  +  v

Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\yasht\OneDrive\Desktop\rock paper scissors project>py main.py
===== ROCK PAPER SCISSORS =====

Choose Difficulty:
1. Easy
2. Medium
3. Hard
Enter your choice: 1
Difficulty: Easy

Enter number of rounds: 2

Round 1
Enter rock, paper, or scissors: hi
Invalid choice!
Please enter rock, paper, or scissors: rock
Your choice: rock
Computer choice: paper
Result: computer wins

Round 2
Enter rock, paper, or scissors: rock
Your choice: rock
Computer choice: paper
Result: computer wins

===== FINAL SCORE =====
Wins: 0
Losses: 2
Draws: 0

===== GAME HISTORY =====
Round 1 : You = rock | Computer = paper | Result = computer wins
Round 2 : You = rock | Computer = paper | Result = computer wins
```

Figure 2: Input validation test. The invalid input "hi" was rejected and the program asked for a valid choice before continuing.

13. Testing Approach

Testing was done by running the program with different numbers of rounds and by checking normal and invalid inputs.

Test	Input / Condition	Observed Result
1	Easy difficulty, 3 rounds	Game completed; 0 wins, 2 losses, 1 draw; history displayed.
2	Invalid input: hi	Program displayed Invalid choice! and requested input again.
3	Valid input after invalid input	Game continued normally after rock was entered.
4	Multiple rounds	Score and history were updated after each round.

14. Challenges Faced

During project setup, the Python files were initially created as empty text files. This caused import errors when the modular program was first run. The issue was fixed by opening each .py file, adding the required code and saving the files with the correct Python extension.

Another small challenge was checking invalid user input. The validation module was tested with an invalid value and the program correctly asked for a valid choice.

15. Learnings and Key Takeaways

- Learned how to divide a Python project into multiple modules.
- Practiced functions and function arguments.
- Used if/elif/else conditions for game logic.
- Used loops for multiple rounds and input validation.
- Used lists and dictionaries for storing data.
- Learned how modules can import functions and variables from other files.
- Learned basic testing by checking both valid and invalid inputs.

16. Future Enhancements

- Add a graphical user interface.
- Add more detailed statistics for previous games.
- Allow the player to restart the game without running the program again.
- Store history permanently in a file.
- Add more computer strategies for difficulty levels.

17. Conclusion

The Rock Paper Scissors project demonstrates a small but complete Python application. It combines game logic, user input, validation, score tracking, history and modular file organization. The project was run successfully from Command Prompt and tested using both normal and invalid inputs.

18. References

1. VITYarthi – Build Your Own Project: General Project Instructions & Submission Guidelines.

2. Python programming concepts covered in the course.